

ACI · AIMLCZG557

Adversarial Search: Minimax & Alpha-Beta

Advanced Computing for AI — Lesson 6

Exam: 28 Jun 2026 · Likely 10–15 marks · Trace the full tree

How to use this companion. Blue = the idea. Amber = everyday analogy. Green = fully-solved example. Red = common mistake. Purple = one line to remember.

1 Game Theory Basics

In adversarial search, two players take turns. Each player is **rational** — they will always pick the best move for themselves. Your agent is the **MAX** player (wants to maximise the utility). The opponent is the **MIN** player (wants to minimise it). We consider two-player, **zero-sum** (one player's gain = other's loss), **perfect-information**, **deterministic** games.

The intuition

You are playing a board game against a perfect opponent. You plan several moves ahead, but you must assume your opponent will always pick the worst possible response for you. The question is: what is your best move, given that assumption?

Formal definition:

- S_0 — initial state (starting board configuration).
- $\text{Player}(s)$ — whose turn it is in state s .
- $\text{Actions}(s)$ — set of legal moves from state s .
- $\text{Result}(s, a)$ — state reached after action a in state s .
- $\text{Terminal-Test}(s)$ — true if the game is over.
- $\text{Utility}(s, p)$ — numeric outcome for player p at terminal state s .

Where this shows up in ML. GANs (Generative Adversarial Networks) are a two-player game: the generator is MAX (maximises discrimination ability), the discriminator is MIN (minimises the generator's chance of fooling it). The training loop is a minimax game.

Key takeaway

Adversarial search: MAX wants the highest utility, MIN wants the lowest. Both play optimally. Zero-sum means what MAX wins, MIN loses.

2 The Minimax Algorithm

Minimax computes the optimal move by working **backwards from the leaves**:

- At **MAX nodes**: return max of children's values.

- At **MIN nodes**: return min of children's values.
- At **terminal nodes**: return the utility value.

Pseudocode:

```
function MINIMAX-VALUE(s):
  if TERMINAL-TEST(s): return UTILITY(s)
  if PLAYER(s) == MAX:
    return max(MINIMAX-VALUE(c) for c in SUCCESSORS(s))
  else:
    return min(MINIMAX-VALUE(c) for c in SUCCESSORS(s))
```

Properties: Complete (yes, if game tree is finite). Optimal (yes, vs optimal opponent). Time: $O(b^m)$. Space: $O(bm)$.

 Worked example: 4-level minimax tree — complete trace

Tree: MAX at root, then MIN, then MAX, then leaf values. Leaves (left to right): 3, 5, 2, 9, 1, 8, 6, 4. Tree structure: root has 2 children (A and B). A has 2 MIN children (A1, A2). B has 2 MIN children (B1, B2). Each MIN child has 2 leaf children.

Step 1. Evaluate MIN node A1. Children = 3, 5. $\min(3, 5) = 3$.

Step 2. Evaluate MIN node A2. Children = 2, 9. $\min(2, 9) = 2$.

Step 3. Evaluate MAX node A. Children (MIN values) = 3, 2. $\max(3, 2) = 3$.

Step 4. Evaluate MIN node B1. Children = 1, 8. $\min(1, 8) = 1$.

Step 5. Evaluate MIN node B2. Children = 6, 4. $\min(6, 4) = 4$.

Step 6. Evaluate MAX node B. Children (MIN values) = 1, 4. $\max(1, 4) = 4$.

Step 7. Evaluate root (MAX). Children = 3, 4. $\max(3, 4) = 4$.

Optimal move: go right (choose B). Guaranteed utility = 4.

Sense-check: going left (A) gives only 3; MIN would steer us to 2 via A2. Going right (B) gives 4 because MIN can only reduce it to 4 at worst (via B2).

 Watch out

MAX cannot simply pick the path to the highest leaf (9 is available under A2). MIN gets to choose at the MIN level and will pick 2, not 9. Always apply the correct operator (max or min) at each level; never confuse which player moves when.

 Key takeaway

Minimax: work bottom-up. MIN nodes take min; MAX nodes take max. The root value is the guaranteed utility for MAX against a perfect opponent.

3 Alpha-Beta Pruning

Alpha-beta pruning improves minimax by skipping branches that cannot possibly affect the final decision. It produces the same result as minimax but often much faster.

 The intuition

Suppose you are MAX and you have already secured a guaranteed value of 3 from the left branch. Now you are exploring the right branch, and MIN already has a move that gives 1. You know MIN will never let you get more than 1 from this branch — so stop exploring it. You already have 3; this branch can't beat that.

Two parameters:

- α = best value MAX can guarantee so far (starts at $-\infty$, increases at MAX nodes).
- β = best value MIN can guarantee so far (starts at $+\infty$, decreases at MIN nodes).

Prune when: $\alpha \geq \beta$.

- At a **MIN node**: if current value $\leq \alpha$, prune (MIN already lower than MAX's guarantee).
- At a **MAX node**: if current value $\geq \beta$, prune (MAX already higher than MIN's guarantee).

 Worked example: Alpha-beta pruning trace — same tree as minimax example

Same tree: root MAX, leaves 3,5,2,9,1,8,6,4. We track (α, β) at each node.

- Step 1. Root (MAX),** $\alpha = -\infty, \beta = +\infty$. **Go left to A.**
- Step 2. Node A (MIN),** $\alpha = -\infty, \beta = +\infty$. **Go to A1.**
- Step 3. Node A1 (MIN),** $\alpha = -\infty, \beta = +\infty$. See leaf 3: $\beta = \min(+\infty, 3) = 3$. See leaf 5: $\beta = \min(3, 5) = 3$. Return 3.
- Step 4. Back at A (MIN):** value from A1 = 3. $\beta = 3$. Update root's $\alpha = \max(-\infty, 3) = 3$. Go to A2.
- Step 5. Node A2 (MIN),** $\alpha = 3, \beta = +\infty$. See leaf 2: $\beta = \min(+\infty, 2) = 2$. Now $\alpha = 3 \geq \beta = 2 \rightarrow$ **PRUNE remaining children of A2**. Return 2.
- Step 6. Node A returns** $\min(3, 2) = 2$. **Root α stays 3.** (Root already has 3 from A1; A gave only 2.)
Wait: root is MAX; it evaluates $\max(3, 2) = 3$ from branch A. Root $\alpha = 3$.
- Step 7. Go right to B (MIN),** $\alpha = 3, \beta = +\infty$. Go to B1.
- Step 8. Node B1 (MIN),** $\alpha = 3, \beta = +\infty$. See leaf 1: $\beta = 1$. Now $\alpha = 3 \geq \beta = 1 \rightarrow$ **PRUNE B1**. Return 1.
- Step 9. Back at B (MIN):** value from B1 = $1 \leq \alpha = 3 \rightarrow$ **PRUNE B entirely** (B2 need not be explored). Return 1.
- Step 10. Root (MAX):** $\max(3, 1) = 3$. **Optimal move: go left (choose A), value = 3.**

Pruned nodes: second leaf of A2, and all of B2. Saved 3 leaf evaluations out of 8.
 Sense-check: same answer as minimax (3), but with less work. Note: in the original 8-leaf example the optimal was 4 (choose B); here the values differ — always work with the actual values given in the exam.

 Watch out

The prune condition is $\alpha \geq \beta$ (alpha *greater than or equal to* beta). Use $\geq$, not $>$. When they are equal, the branch cannot improve the result and may be pruned. Also: pruning does NOT change the minimax value — it only saves computation.

Alpha-beta efficiency. With *perfect* move ordering (best moves explored first), alpha-beta reduces the effective branching factor from b to $b^{1/2}$, cutting time from $O(b^m)$ to $O(b^{m/2})$ — doubling the searchable depth for the same computation.

Where this shows up in ML. Alpha-beta is used in game-playing engines (chess, Go). AlphaGo replaced it with MCTS, but the prune-and-evaluate principle reappears in beam search decoding and branch-and-bound optimisation.

✓ Key takeaway

Alpha-beta: α = MAX's floor, β = MIN's ceiling. Prune when $\alpha \geq \beta$. Same result as minimax, up to $O(b^{m/2})$ faster with good ordering.

4 Monte Carlo Tree Search (MCTS)

MCTS avoids evaluating every node. Instead it **simulates** random games from a node to estimate its value. It balances exploration (visiting new nodes) and exploitation (revisiting good nodes) using the **UCB1 formula**:

$$\text{UCB1}(n) = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}$$

where w_i = wins from node i , n_i = visits to node i , N = total visits to parent, c = exploration constant (typically $\sqrt{2}$).

Four phases of MCTS:

1. **Selection.** Walk down the tree, always picking the child with highest UCB1, until a leaf or unexpanded node.
2. **Expansion.** Add one or more child nodes.
3. **Simulation.** Play out a random game from the new node to a terminal state.
4. **Backpropagation.** Update w_i and n_i for every node on the path.

👉 The intuition

Instead of solving the whole tree analytically, MCTS sends scouts on random expeditions. Each scout reports back on what they found. Over time, the paths that keep yielding good results get explored more. The UCB1 formula is the rule for deciding where to send the next scout.

Where this shows up in ML. AlphaGo and AlphaZero use MCTS with a value network (replaces random simulation) and a policy network (guides selection). This combination is why they surpassed classical minimax engines.

✓ Key takeaway

MCTS: Select (UCB1) → Expand → Simulate (random rollout) → Backpropagate. No need for a heuristic evaluation function; accuracy improves with more simulations.

5 Utility Functions & Static Evaluation

In real games like chess, we cannot search to terminal states (the tree is too deep). We cut off at a fixed depth and use a **static evaluation function** (also called a **utility function** or **heuristic evaluation**) to estimate the value of a non-terminal state.

- Chess: material count, piece mobility, king safety, pawn structure.
- Tic-tac-toe: number of winning lines open to MAX minus lines open to MIN.

Quiescence search: avoid the *horizon effect* by not cutting off in the middle of a “noisy” sequence (captures, checks). Continue searching until the position is stable (quiescent).

✗ Watch out

The evaluation function must agree with the terminal utility for terminal states. If the evaluation function is too simplistic, the agent may make bad decisions near the horizon — this is called the horizon effect. Quiescence search is the standard fix.

✓ Key takeaway

Static evaluation = depth-limited minimax + heuristic at cutoff. Quality of play depends heavily on the quality of the evaluation function.